

Linux RAM Disk & Optimization Guide

This guide details how to create and utilize RAM disks on Linux to improve application performance (e.g., for gaming shaders, browser caches, or temporary build artifacts).

1. What is a RAM Disk?

A RAM disk acts as a virtual hard drive that resides entirely in your computer's Random Access Memory (RAM). * **Speed:** It is significantly faster than SSDs or HDDs. * **Volatility:** All data is lost when the computer is turned off or restarted. * **Use Case:** Ideal for temporary files, cache, or specific application folders that require high-speed access (like shader caches in emulators).

2. Method A: The /dev/shm Method (Rootless/Easy)

Linux distributions typically mount a generic shared memory filesystem at /dev/shm. This is the easiest way to use a RAM disk because it usually doesn't require sudo to write files (unlike mounting a new filesystem).

Pros: No root privileges needed for the user; efficient.

Cons: Shared with other system processes; size limited to half of physical RAM by default.

Usage

You can simply create a directory inside /dev/shm:

```
mkdir /dev/shm/my_fast_folder
```

You can now write files to /dev/shm/my_fast_folder, and they will be stored in RAM.

3. Method B: The Manual tmpfs Mount (Root/Custom Size)

If you need a dedicated RAM disk with a specific size or mount point, use the mount command.

Pros: Dedicated space; customizable size limits.

Cons: Requires sudo to mount/unmount.

Step-by-Step

1. **Create a Mount Point:** `bash        sudo mkdir -p /mnt/ramdisk`
 2. **Mount the RAM Disk:** Use tmpfs (Temporary File System). Here we allocate 4GB: `bash        sudo mount -t tmpfs -o size=4G tmpfs /mnt/ramdisk`
 3. **Set Permissions:** Allow your user to write to it: `bash        sudo chown $USER:$USER /mnt/ramdisk`
 4. **Verify:** `bash        df -h /mnt/ramdisk`
-

4. Understanding RAM Usage (Dynamic vs. Fixed)

A common misconception is that creating a 4GB RAM disk immediately consumes 4GB of your physical RAM. This is **incorrect** for both methods described above.

Linux tmpfs (used by both `/dev/shm` and the manual method) is **dynamic**:

1. **It Only Consumes What You Store:** If you create a “4GB” RAM disk but only copy **100MB** of files into it, it will only use **100MB** of your physical RAM. The rest remains available for your system to use as normal memory.
2. **The “Size” is a Limit, Not a Reservation:**
 - **Manual Mount (size=4G):** You are setting a *ceiling*. You tell Linux, “Do not allow this specific folder to grow larger than 4GB.”
 - **/dev/shm:** By default, this is usually capped at **50% of your total physical RAM**. (e.g., on a 16GB system, it can grow up to 8GB, but starts at 0 bytes).

Why specify a size manually? Since `/dev/shm` is already available, users typically only use the manual method to *restrict* an application (e.g., “Stop this cache from eating all my RAM”) or to ensure a dedicated space separate from other system processes.

5. Redirecting Folders with Symbolic Links

Applications often expect files to be in specific locations (e.g., `~/ .local/share/app/cache`). To move this data to RAM without breaking the application, we use a **Symbolic Link (symlink)**.

The Logic

1. **Move** the original data to the RAM disk.

2. **Delete** the original folder.
3. **Link** the RAM disk location back to the original path.

Example: Speeding up “Citron” Shaders

Scenario: Citron stores shaders at `~/.local/share/citron/shader`. We want this in RAM.

Script:

```
#!/bin/bash

ORIGINAL_DIR="$HOME/.local/share/citron/shader"
RAM_DIR="/dev/shm/citron_shader_cache"
BACKUP_DIR="$HOME/citron_shader_backup" # For saving to disk later

# 1. Create the RAM directory
mkdir -p "$RAM_DIR"

# 2. If the original is a real folder, move it to backup, then copy
to RAM
if [ -d "$ORIGINAL_DIR" ] && [ ! -L "$ORIGINAL_DIR" ]; then
    mkdir -p "$BACKUP_DIR"
    cp -r "$ORIGINAL_DIR"/* "$BACKUP_DIR/"
    cp -r "$ORIGINAL_DIR"/* "$RAM_DIR/"
    rm -rf "$ORIGINAL_DIR"

    # 3. Create the Symlink
    ln -s "$RAM_DIR" "$ORIGINAL_DIR"
    echo "Symlink created! $ORIGINAL_DIR -> $RAM_DIR"

# 4. If it's already a link, just refresh RAM from backup
elif [ -L "$ORIGINAL_DIR" ]; then
    if [ -d "$BACKUP_DIR" ]; then
        cp -r "$BACKUP_DIR"/* "$RAM_DIR/"
    fi
    echo "RAM cache refreshed."
fi
```

6. Persistence (Saving Data)

Because RAM is volatile, you must copy data back to the disk before shutting down if you want to keep it.

Save Script Example:

```
#!/bin/bash

RAM_DIR="/dev/shm/citron_shader_cache"
BACKUP_DIR="$HOME/citron_shader_backup"

if [ -d "$RAM_DIR" ]; then
    mkdir -p "$BACKUP_DIR"
    # Sync data from RAM back to SSD
    cp -r "$RAM_DIR"/* "$BACKUP_DIR/"
    echo "Saved to disk."
fi
```